Python 3.12 — asyncio & High-Performance Coroutines

Source basis: Python 3.12 standard-library documentation

Learning goals

- Explain core concepts and when they are useful.
- Read and adapt code/query examples.
- Recognize trade-offs, common errors, and verification steps.

How to use this chapter

Read the concept first, run examples in an isolated environment, then use the official references at the end of the chapter to verify version-specific behavior. The examples are teaching samples, not production-ready systems.

1. Scope and mental model

asyncio is Python's standard-library framework for asynchronous I/O. async def defines coroutine functions, await suspends a coroutine, and an event loop schedules tasks.

It is useful for I/O-bound programs with many concurrent operations: network clients, servers, sockets, queues, and coordination of async libraries. It does not automatically parallelize CPU-heavy code.

Concurrency means tasks make progress during overlapping periods; parallelism means simultaneous execution. asyncio commonly switches tasks on one event-loop thread at await points.

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

2. Coroutines, awaitables, and tasks

Calling an async function creates a coroutine object; it does not run to completion on its own. Awaiting the coroutine or scheduling it as a Task lets the event loop execute it.

asyncio.run() is a recommended top-level entry point. Do not call it inside a thread that already has a running event loop. Use asyncio.sleep() rather than time.sleep() for asynchronous delays.

Example code / SQL

```
import asyncio

async def greet(name: str) -> str:
    await asyncio.sleep(0.1)
    return f"Hello, {name}"

async def main():
    print(await greet("Ada"))

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

3. Scheduling concurrent tasks

asyncio.create_task() schedules a coroutine to run concurrently with the current coroutine. asyncio.gather() awaits multiple awaitables and returns results in input order.

Keep track of background tasks and await or cancel them during shutdown. CPU-heavy code without await points can monopolize the event loop and delay unrelated tasks.

Example code / SQL

```
import asyncio

async def work(label, delay):
    print("start", label)
    await asyncio.sleep(delay)
    return label + " done"

async def main():
    tasks = [asyncio.create_task(work("A", .3)),
             asyncio.create_task(work("B", .1))]
    print(await asyncio.gather(*tasks))

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

4. Timeouts and cancellation

Cancellation is part of normal asynchronous control flow. task.cancel() requests cancellation, which is raised at a cancellation point. Use try/finally for cleanup and generally re-raise CancelledError after cleanup.

Python 3.12 provides asyncio.timeout() and asyncio.wait_for(). Set deadlines for external operations and avoid waits that can continue indefinitely.

Example code / SQL

```
import asyncio

async def slow():
    try:
        await asyncio.sleep(10)
    finally:
        print("cleanup runs")

async def main():
    try:
        async with asyncio.timeout(0.2):
            await slow()
    except TimeoutError:
        print("deadline exceeded")

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

5. Bounded concurrency

Creating thousands of tasks can overload a remote service or consume memory. A semaphore limits how many tasks enter a protected section at once; it is a form of backpressure, not CPU parallelism.

Use async with to guarantee semaphore release after errors or cancellation. For queued workloads, asyncio.Queue(maxsize=...) can bound backlog.

Example code / SQL

```
import asyncio

async def request(i, limit):
    async with limit:
        await asyncio.sleep(.05)
        return f"result-{i}"

async def main():
    limit = asyncio.Semaphore(5)
    results = await asyncio.gather(
        *(request(i, limit) for i in range(30)))
    print("completed:", len(results))

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

6. Producer-consumer queues

asyncio.Queue coordinates asynchronous producers and consumers. A bounded queue applies backpressure because put() waits when the queue is full.

Workers need a shutdown protocol. One simple approach sends one sentinel per worker after producers finish. Call task_done() for each item removed if the program uses queue.join().

Example code / SQL

```
import asyncio
STOP = None

async def worker(queue):
    while True:
        item = await queue.get()
        try:
            if item is STOP:
                return
            await asyncio.sleep(.03)
            print("processed", item)
        finally:
            queue.task_done()

async def main():
    q = asyncio.Queue(maxsize=4)
    workers = [asyncio.create_task(worker(q)) for _ in range(2)]
    for i in range(8):
        await q.put(i)
    for _ in workers:
        await q.put(STOP)
    await q.join()
    await asyncio.gather(*workers)

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

7. Async network streams

asyncio streams provide high-level TCP APIs. open_connection() returns a reader and writer. writer.drain() supports flow control when buffered output grows.

TCP is a byte stream and does not preserve application message boundaries. Define framing, maximum sizes, encoding, authentication, and error handling for real protocols.

Close writers and await wait_closed() where appropriate. Add timeouts and handle connection failures.

Example code / SQL

```
import asyncio

async def main():
    reader, writer = await asyncio.open_connection("example.com", 80)
    writer.write(b"GET / HTTP/1.1\r\nHost: example.com\r\n"
                 b"Connection: close\r\n\r\n")
    await writer.drain()
    print((await reader.readline()).decode(errors="replace").strip())
    writer.close()
    await writer.wait_closed()

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

8. Blocking and CPU-bound work

Never run long blocking calls directly on the event-loop thread. asyncio.to_thread() can move blocking functions to a worker thread and is mainly useful for I/O-bound functions.

CPU-heavy Python code may block the event loop. Consider a process pool or separate worker service, and measure the overhead before choosing. A synchronous HTTP client called inside async def can block all other tasks.

Example code / SQL

```
import asyncio
import time

def blocking_io():
    time.sleep(.3)
    return "completed"

async def main():
    result = await asyncio.to_thread(blocking_io)
    print(result)

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

9. Debugging, testing, and performance

Enable asyncio debug mode in development to identify slow callbacks, forgotten awaits, and resource warnings. Use task names and structured logging to understand concurrent execution.

Test success, exception, timeout, cancellation, empty inputs, load, and shutdown. Measure end-to-end throughput and latency with the same workload when comparing approaches.

Use async-compatible libraries throughout the call path so synchronous blocking calls do not stall the event loop.

Example code / SQL

```
import asyncio

async def square(i):
    await asyncio.sleep(.01)
    return i * i

async def main():
    results = await asyncio.gather(*(square(i) for i in range(100)))
    assert len(results) == 100 and results[7] == 49
    print("checks passed")

asyncio.run(main())
```

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

10. Review checklist and exercises

Checklist: await coroutines; manage task lifetimes; set timeouts; bound concurrency; clean up on cancellation; avoid blocking the event loop; and test shutdown paths.

Exercises: compare sequential awaits with gather(); cap concurrency at three; cancel a task and verify finally runs; and replace an unbounded queue with Queue(maxsize=5).

Review questions: Why does calling an async function not complete its work? What does a semaphore limit? Why is cancellation cleanup important?

Implementation and review notes

Keep the event loop free of blocking calls. Set deadlines, handle cancellation deliberately, bound concurrency, and ensure tasks are awaited or cancelled during shutdown.

Official references and further reading

These trusted official sources are the authority for version-specific details. This PDF summarizes key ideas; consult the linked references for full API specifications and additional examples.

Python 3.12: asyncio overview
https://docs.python.org/3.12/library/asyncio.html

Python 3.12: Coroutines and Tasks
https://docs.python.org/3.12/library/asyncio-task.html

Python 3.12: Synchronization primitives
https://docs.python.org/3.12/library/asyncio-sync.html

Python 3.12: Streams
https://docs.python.org/3.12/library/asyncio-stream.html

Python 3.12: Event loop
https://docs.python.org/3.12/library/asyncio-eventloop.html

Suggested practice checklist

- Confirm the runtime/version.
- Run the example in an isolated project or disposable database.
- Test one normal case and one failure/edge case.
- Record commands, expected output, and any assumptions.